UAV-BASED DEPLOYMENT SYSTEM FOR RAPID ESTABLISHMENT OF EMERGENCY COMMUNICATION NETWORKS

PROJECT REPORT

Submitted in partial fulfillment of the
requirement for the award of the
Degree of

BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMPUTER ENGINEERING

by

RONIE SAMUEL (23BLC1061)
ANMOL KAPIL (23BLC1229)
GARV RAWAT (23BLC1058)

Under the Guidance of

DR. BIBHUTI BHUSAN PRADHAN



SCHOOL OF ELECTRONICS ENGINEERING
VELLORE INSTITUTE OF TECHNOLOGY
CHENNAI - 600127

October 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

Declaration

We hereby declare that the report titled ***UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks*** submitted by us to the School of Electronics Engineering, Vellore Institute of Technology, Chennai, in partial fulfilment of the requirements for the award of **Bachelor of Technology** in **Electronics and Computer Engineering**, is a bonafide record of the work carried out by us under the supervision of ***Dr. Bibhuti Bhusan Pradhan, Associate Professor, SENSE***.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Signature:	Signature:	Signature: . . . . .
Name: Ronie Samuel	Name: Anmol Kapil	Name: Garv Rawat
Reg. No.: 23BLC1061	Reg. No.: 23BLC1229	Reg. No.: 23BLC1058
Date:	Date:	Date: . . . . .

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Electronics Engineering

Certificate

This is to certify that the project report titled ***UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks*** submitted by **Ronie Samuel (23BLC1061), Anmol Kapil (23BLC1229), Garv Rawat (23BLC1058)** to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Electronics and Computer Engineering** is a bonafide work carried out under my supervision. The project report fulfils the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor	**Head of the Department**
Signature:	Signature: .....
Name: Dr. Bibhuti Bhusan Pradhan	Name: .....
Date:	Date: .....

Examiner 1	**Examiner 2**
Signature:	Signature: .....
Name:	Name: .....
Date:	Date: .....

(Seal of the School)

Abstract

Earthquakes, cyclones, floods and other forces regularly split apart the ground based telecommunication towers, fibre-optic trunks & electrical distribution lines. This results in communication interruptions on a large scale during critical coordination times of emergencies. Disaster recovery systems (vehicular cellular towers, satellite terminals) are time-consuming, expensive and may not be able to cover over rugged terrain. This project presents a UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks consisting of a physical Unmanned Aerial Vehicle (UAV), a custom laptop-based Ground Control Station (GCS) with a Python backend, automated spatial Radio Frequency (RF) coverage mapping and a servo-actuated physical node-deployment mechanism, supported by preliminary algorithm development in MATLAB/Simulink.

The F450X frame and brushless DC propulsion system send and receive data to and from the laptop based GCS using a wireless radio telemetry link over the open source MAVLink v2.0 protocol, implemented on the ArduPilot firmware running on an Edith IronCore F4 autopilot (microcontroller STM32F405R). Vehicle telemetry data (attitude, GPS position, altitude, battery state, flight modes) is transmitted to the GCS where it is used to send commands to the vehicle such as: flight mode change, loiter hold, automatic waypoint missions, and return to launch (RTL). There is a 5 V isolated power switcher built into the UAV under the chassis that is controlled by a flight controller PWM channel auxiliary and a dedicated servo actuated payload release mechanism.

The RF coverage surveying and decision-making pipeline is implemented in the Ground Control Station, using the operational Python code in the backend that is implemented asynchronously, with the FastAPI framework. The system gets RSSI from the GPS telemetry stream of the flight controller and geographic positions (latitude, longitude and altitude). The back-end of the Python process analyzes the data stream and generates continuous coverage heat maps and communication category classes of the surveyed area (good, moderate, weak and dead zones). The algorithm is an automatic site-scoring algorithm which takes into account the connectivity requirement and distance to remaining alive anchor nodes to guarantee backhaul reachability with the desired area of coverage recovery. Once the optimum site is identified it will be converted into a GPS waypoint and sent out to the drone to be deployed.

MATLAB/Simulink modeling of 2.4 GHz radio propagation including shadow fading and use of software packages for spatial interpolation and site scoring to exercise the software versions of these functions prior to integration in the field. Stable physical flight of the UAVs has been successfully demonstrated and the MAVLink packet delivery distance in the experimental field test has reached 98.4% mean, while the dropping of the servo payload has reached 4.3 m mean GPS waypoint repeatability. According to the preliminary simulation results, the overall coverage improvement after the nodes deployment based on this algorithm was 16.3 percentage points (from 47.5% to 63.8%). The system developed is technically feasible closed loop system that combines the aerial RF reconnaissance ability with the targeted physical payload deployment in humanitarian disaster relief.

Keywords: Unmanned Aerial Vehicle (UAV), Emergency Communication, Ground Control Station (GCS), Python Backend, FastAPI, MAVLink, RSSI Mapping, Inverse Distance Weighting (IDW), Site Scoring, Servo Release, MATLAB/Simulink.

Acknowledgements

We wish to express our sincere gratitude to our project supervisor, **Dr. Bibhuti Bhusan Pradhan**, Associate Professor, School of Electronics Engineering (SENSE), Vellore Institute of Technology, Chennai, for his continuous guidance, technical insights, and encouraging support throughout the entire duration of this project.

We express our heartfelt appreciation to **Dr. Ravi Sankar A**, Dean, School of Electronics Engineering (SENSE); **Dr. K. Mohanaprasad**, Associate Dean (Academics), SENSE; and **Dr. Chitra K**, Associate Dean (Research), SENSE, VIT Chennai, for providing excellent academic infrastructure, laboratory resources, and encouragement for practical research.

We extend our sincere thanks to the Head of the Department, Electronics and Communication Engineering / Electronics and Computer Engineering, SENSE, VIT Chennai, for administrative support and facilitating the smooth progression of this capstone work.

We gratefully acknowledge the Final Year Project Coordination Team — Dr. S. Markkandan, Dr. Anisha Natarajan, Dr. Sukriti, and Dr. Shaik Ahmadsaidulu — for their structured coordination, continuous evaluation, and timely guidance across all project reviews.

Finally, we express our deepest gratitude to our parents, family members, and friends for their enduring patience, moral support, and encouragement throughout our academic programme.

Codes and Standards

Project Title: *UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks*

The project is designed to develop a system which can be deployed using UAVs to quickly establish an emergency communication network.

1. **Standards for UAV / Aerial Systems**
    - **DGCA (India) UAS Rules 2021:** Unmanned Aircraft Systems (UAS) Rules in India issued by Directorate of General Aviation (DGCA). Explain the concept of airworthiness, visual line of sight operations (VLOS), altitude limits (< 120 m AGL) and operator safety procedures.
    - **MAVLink v2.0 Micro Air Vehicle Protocol:** Lightweight binary serialization protocol for bi-directional (ground-to-air) telemetry, autopilot status reporting, mission upload/download and flight mode switching, and sending of auxiliary servo commands.
2. **Wireless Communication Standards**
    - **Wireless networks (2.4 GHz WLAN):** Preferred means of communication in emergency, there is a common standard. IEEE 802.11 b/g/n is a standard that uses a 2.4 GHz frequency. A series of simulations were run to model the radio waves in this range of frequencies to determine its effectiveness. With these simulations, the loss of signal with distance was determined and the strength of the signals received was determined as a reference. This information is very critical in developing good emergency communication.
    - **IEEE 802.11s (Wireless Mesh Networking):** Architecture reference for self-configuring multi-hop emergency mesh communications between ground capsules deployed and relief command centres.
3. **Standards for the Safety of the Hardware / Actuation**
    - **IEC 62133 (Lithium Secondary Cells/Batteries Safety):** High discharge LiPo battery packs for UAV propulsion, electrical systems (ESCs) and avionics are covered by this electrical and thermal safety standard.
    - **Standard 50 Hz PWM Servo Actuation Conventions (1000 μs locked, 2000 μs released):** Were used for the PWM actuated payload latch (via the flight controller's auxiliary rail).
    - **Manual Flight Response using Standard SBUS Serial Receiver Protocol:** Inverted asynchronous serial framing (100,000 bps, 8E2) on the ESP32 receiver interface and flight control.
4. **Emergency Telecommunication References**
    - **ITU-R M.1042 (Temporary Radio Sets):** Requirements for temporary radio sets in disaster situations.
5. **Architecture of Software / Web Standards**
    - **FastAPI REST / OpenAPI Specifications (RFC 7231 / OpenAPI 3.0):** The node inventory management, survey triggering and mission sending back-end for FastAPI (via REST / OpenAPI Specifications (RFC 7231 / OpenAPI 3.0)), Architecture of stateless HTTP endpoints.
    - **WebSocket Protocol (RFC 6455) by W3C:** Low Latency Bidirectional Communication for Real-time streaming of UAV telemetry data and live RSSI samples from the back-end to the Leaflet web interface.
6. **Simulation / Prototyping Guidelines**

- **MathWorks MATLAB/Simulink System Modeling Guidelines:** Conventions to observe when configuring the discrete-time simulation blocks, log-distance path-loss and candidate site-scoring routines in initial algorithm development.

Mapping of Sustainable Development Goals (SDGs)

Project Title: UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks

Project Title	Mapped SDGs	Justification	Example Impact
UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks	**SDG 9** – Industry, Innovation and Infrastructure	This entails the utilization of both physical UAV flight as well as Python-based GCS telemetry, automated RSSI spatial analysis and servo payload deployment processes towards the restoration of damaged communications.	This could help to re-establish more resilient temporary communications infrastructural facilities for use in remote locations where cellular towers have been knocked down.
	SDG 11 – Sustainable Cities and Communities	The system can help to identify isolated pockets of connectivity while also accurately deploying emergency communication nodes to help support rescue logistics operations.	This can make a big difference in how we handle disasters. It will help organizations that deal with emergencies, like those that respond to floods and landslides in cities and rural areas. These groups will be better able to react and help people when disasters strike.
	SDG 13 – Climate Action	This refers to providing rapidly-deployable battery-powered low-emission aerial reconnaissance and networking systems that can help provide much-needed communication support following climate-related disasters.	This will help the affected communities to quickly access aerial reconnaissance and communication services during climate-related disaster scenarios.
	SDG 17 – Partnerships for the Goals	This involves the use of open-source protocols including MAVLink v2.0, FastAPI REST and WebSockets as well as open-hardware UAV standards towards achieving greater levels of system interoperability.	This can make a big difference. It will allow us to create modules that can work on their own to design mesh nodes, which is really helpful. Plus, it will also support platforms that help people in need at the agency level. This means we can use these modules to make a positive impact in many different areas.

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

Codes and Standards v

Mapping of Sustainable Development Goals (SDGs) vii

List of Figures xii

List of Tables xiii

List of Acronyms xiv

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 3
- 1.4 Scope of the Work . . . 4
- 1.5 Organisation of the Report . . . 5

2 Literature Survey 6
- 2.1 Review of Prior Work . . . 6
  - 2.1.1 UAV-Assisted Emergency Communication Networks . . . 6
  - 2.1.2 Airborne RF Mapping and Coverage Characterisation . . . 7
  - 2.1.3 Deployment Optimisation and Site Selection . . . 7
  - 2.1.4 Physical Payload Deployment Systems . . . 8
- 2.2 Gaps Identified . . . 8

3 Methodology 10
- 3.1 System Architecture . . . 10
  - 3.1.1 Operational Workflow . . . 11
- 3.2 Physical UAV Platform and Avionics . . . 12

3.2.1 Airframe and Propulsion System . . . 12
3.2.2 Autopilot Flight Controller and Sensors . . . 12
3.2.3 Remote Control and Signal Conversion Interface . . . 13
3.3 Servo-Actuated Payload Release Mechanism . . . 14
3.4 Ground Control Station and Python Backend Architecture . . . 14
3.4.1 Asynchronous FastAPI Backend . . . 14
3.4.2 REST Endpoints and Communication Interface . . . 15
3.4.3 Interactive Web Frontend (Leaflet.js) . . . 15
3.5 RF Surveying, RSSI Ingestion, and Spatial Mapping . . . 16
3.5.1 Geo-Referenced RSSI Ingestion Pipeline . . . 16
3.5.2 Coordinate Transformation to Local Cartesian Grid . . . 16
3.5.3 Inverse Distance Weighting (IDW) Spatial Interpolation . . . 17
3.5.4 Coverage Tier Classification and Gap Detection . . . 17
3.6 Deployment-Site Scoring Algorithm . . . 17
3.7 Preliminary Simulation and Prototyping in MATLAB/Simulink . . . 18
3.7.1 Role of MATLAB/Simulink in the Project . . . 18
3.7.2 Log-Distance Radio Propagation and Shadow Fading Model . . . 19
3.7.3 Simulink Subsystem Architecture . . . 19
3.8 Software-in-the-Loop Simulation and GCS Integration . . . 19
3.9 Experimental and Testing Setup . . . 21

4 Results and Discussion **23**
4.1 Physical Prototype Evaluation . . . 23
4.1.1 GCS and Flight-Controller Telemetry Performance . . . 23
4.1.2 UAV Flight Demonstration and Waypoint Repeatability . . . 24
4.1.3 Servo-Operated Payload Release Evaluation . . . 24
4.2 GCS Python Backend and Processing Evaluation . . . 25
4.2.1 Telemetry Streaming and Ingestion Throughput . . . 25
4.2.2 Algorithmic Execution Benchmarks . . . 25
4.3 Preliminary MATLAB/Simulink Simulation Results . . . 26
4.3.1 Simulation Environment Configuration . . . 26
4.3.2 Simulated RF Coverage Mapping, Gap Analysis, and Candidate Placement 26
4.3.3 Continuous IDW Heatmap and Normalized Site Scoring . . . 28
4.3.4 Simulated Before-and-After Coverage Recovery . . . 29
4.4 Discussion, Limitations, and Conclusion . . . 29
4.4.1 Comparison with Related Works . . . 29
4.4.2 Engineering Challenges and Practical Solutions . . . 29
4.4.3 Limitations and Future Scope . . . 30
4.5 Conclusion . . . 30

A Appendix **32**
A.1 Python GCS Backend Implementation . . . 32
A.2 Python Multi-Criteria Site-Scoring Algorithm . . . 33
A.3 Flight Controller Hardware and Actuator Interfaces . . . 34
A.3.1 Servo Payload Release Channel Interface . . . 34
A.3.2 ESP32 AFHDS-to-SBUS Signal Converter Interface . . . 35
A.4 Preliminary MATLAB/Simulink Simulation Scripts . . . 35

A.4.1 MATLAB Simulation Script: RF Propagation and IDW Heatmap . . . 35
A.4.2 MATLAB GCS Integration Bridge Script (`setupGCS.m`) 36
A.4.3 Simulink Model Top-Level Subsystem Organization 36

Bibliography **38**

Biodata **38**

List of Figures

3.1 Overall System Architecture: Physical UAV Subsystem, Bidirectional MAVLink Telemetry Link, Python FastAPI Ground Control Station, and Preliminary MATLAB/Simulink Simulation Environment 11
3.2 Operational Workflow: From Disaster Zone Definition and Aerial RF Surveying to Multi-Criteria Candidate Site Selection and Servo-Actuated Node Drop . . . 12
3.3 Physical UAV Prototype: Custom F450X Quadcopter Chassis, Edith IronCore F4 Autopilot, BLDC Motors, 3S LiPo Battery, and Under-Chassis Payload Release Latch 13
3.4 Custom Python FastAPI + Leaflet Ground Control Station Dashboard: Live Telemetry Gauges, RF Survey Heatmap Rendering, and Automated Payload Drop Command Interface 16
3.5 Simulink Model Architecture (EmergencyNetwork.slx): Block Subsystem Interconnections for Ground Nodes, Drone Survey Path, Log-Distance RF Propagation, Coverage Classification, Spatial Gap Detection, Candidate Site Scoring, Node Deployment, and Before/After Connectivity Comparison 20
3.6 Simulink Preliminary Simulation Environment and Initial Coverage Status: (Left) Survey flight tracks and deployed node positions before deployment; (Right) Real-time coverage classification displaying drone location, COMM-003 (-80 dBm, Weak tier), and COMM-001 (-93 dBm, Gap tier) 20
3.7 Software-in-the-Loop (SITL) Gazebo Simulation and Custom Web GCS Integration: (a) Disaster polygon delineation and survey plan generation, (b) In-flight autonomous survey execution with real-time telemetry streaming, and (c) Survey mission completion, landing detection, and spatial gap analysis trigger . 21

4.1 Simulink Dynamic RF Coverage Simulation Outputs: (Left) Real RF Survey Coverage Map (RFRangeScale = 0.30) showing raster survey flight path and active node coverage circles; (Right) Gap Analysis and Ranked Candidate Placement displaying 112 identified coverage gap points and ranked candidate locations. 27
4.2 MATLAB Simulation Execution Output: Survey Sample Classification Breakdown (503 Points: 50 Moderate, 341 Weak, 112 Gap) and Ranked Candidate Coordinates with Computed Suitability Scores. 27
4.3 Spatial RSSI Coverage Heatmap Generated via Inverse Distance Weighting (IDW) Showing Surviving Anchor Base, Survey Tracks, and Candidate Deployment Locations. 28

List of Tables

3.1 Payload Release Mechanism Engineering Specifications 14
3.2 Primary Python FastAPI GCS Endpoints 15
3.3 Wireless Coverage Quality Tier Classification 17

4.1 Physical UAV Autonomous Waypoint Arrival Repeatability (n = 25 Trials) . . . 24
4.2 Servo Payload Release Mechanism Performance Metrics 25
4.3 GCS Backend Algorithmic Execution Latency Breakdown 25
4.4 Preliminary MATLAB/Simulink RF Simulation Parameters 26
4.5 Candidate Deployment Site Multi-Criteria Scoring Breakdown 29
4.6 Simulated Coverage Comparison: Pre- and Post-Deployment Recovery 29
4.7 Benchmarking Proposed System Against Related Works 29

A.1 Flight Controller Auxiliary Servo Channel Connection 34
A.2 ESP32 AFHDS-to-SBUS Signal Converter Pin Connections 35

List of Acronyms

UAV	Unmanned Aerial Vehicle
GCS	Ground Control Station
RF	Radio Frequency
RSSI	Received Signal Strength Indicator
GPS	Global Positioning System
MAVLink	Micro Air Vehicle Link
API	Application Programming Interface
REST	Representational State Transfer
WS	WebSocket
IDW	Inverse Distance Weighting
IMU	Inertial Measurement Unit
ESC	Electronic Speed Controller
BLDC	Brushless Direct Current
LiPo	Lithium Polymer
BEC	Battery Eliminator Circuit
PWM	Pulse Width Modulation
PDB	Power Distribution Board
SBUS	Serial Bus (Remote Control Receiver Protocol)
AFHDS	Automatic Frequency Hopping Digital System
RTL	Return to Launch
SITL	Software In The Loop
SDG	Sustainable Development Goal
DGCA	Directorate General of Civil Aviation
VLOS	Visual Line Of Sight

Chapter 1

Introduction

1.1 Background and Motivation

Wireless communication infrastructure is vital to Modern Society's disaster relief management, emergency medical services, logistics, and public safety. Where strong natural disasters like earthquakes, cyclones, flash floods and landslides hit, they are known to wreak massive damage on the telecommunication towers, cell base transceiver sites, fibre trunk lines and electrical grids simultaneously. This loss of terrestrial communication systems is a disaster communication blackout time which is the golden time during disaster response when emergency operation centers, first responders and trapped civilians have to communicate well in time and regularly to save human lives.

The typical approach to restoring end-to-end communications during disasters is to mount cellular systems on wheels, also known as Cells on Wheels (COWs) or portable mast repeaters, and/or satellite hubs. But there are several weaknesses in each of these systems that hinder quick response and/or recovery in disasters:

- **Slow mobilization:** Equipment like cellular phones, portable repeaters and satellite terminal equipment could take days to move due to rough terrain, washed out bridges, or water logged areas.
- **High COGS and COS:** Commercial satellite terminals and COWs are expensive, and COGS and COS is prohibitively high.
- **Disaster response operations are also hindered:** Cellular signals have a very limited range and cannot penetrate dense foliage or man-made urban structures.

Time between when first responders are able to deploy and establish communication systems is the wrong time in a disaster. The much touted golden 72 hours in disaster response is thus wasted in getting spotters on a roof top, installing COWs on a rocky ground, launching satellites in orbit etc. Meanwhile, the disaster response operation is delayed since emergency managers are in the dark about the disaster scene and trapped people.

The UAVs can be deployed quickly for offering an aerial observation platform to restore much needed communication during disasters. Small multirotor UAVs can rapidly be deployed by a

field team, traverse rough terrain, and provide a first responder with a line of sight (LoS) radio link at a low altitude. An UAV-based disaster response system which utilizes UAVs as airborne relay, base stations, and radio reconnaissance resources has been suggested. But there are a few drawbacks in the existing solutions.

In general, current solutions don't take into account a UAV as a communication node with reconnaissance and precision deployment capacities during a disaster. The role of the UAV in the current solutions is predominantly either as an airborne communication repeater, which becomes unusable when power runs out, or as a cargo transport UAV, dropping cargo at arbitrary locations without knowledge of the radio environment. The UAVs have been shown to be highly useful to disaster response by delivering aerial imaging and situational awareness, but not so effective in delivering disaster communication assistance. The project aims to offer an integrated solution for disaster response using UAV resources in optimal way, with a communication link between the UAV operator and the GCS, and with the deployment of communication nodes in optimal location depending on the radio environment.

The system that is proposed consists of:

1. A multi rotor UAV platform with manual and waypoint flight modes, using open-source flight controllers and wirelessly communicating with a laptop ground control station.
2. A physical capsule with communication nodes that is deployed using a servo deployment mechanism.
3. An operational RF coverage surveying and Python-based GCS processing pipeline, where the UAV survey flights ingest geotagged RSSI values, the coverage is represented as a heatmap based on inverse distance weighting (IDW), the coverage gaps are segmented and optimal deploy locations are suggested by multi-criteria decision analysis approach.
4. A MATLAB/Simulink model used in the development process to test the log-distance path-loss model and the scoring algorithm for the selection of the deployment location for an UAV.

1.2 Problem Statement

When the existing communication structure breaks down due to any catastrophic disaster, emergency response teams require a situation in which they can immediately know where the communication is missing, and then deploy more communication relays to restore the communication. The existing UAV-based emergency communication methods have three structural limitations:

- **In a traditional UAV relay system,** the UAV needs to remain in the air to maintain coverage, the limited endurance of common multi-rotor aircraft (15–25 minutes) makes the coverage disappear as soon as the UAV must land for recharge. A "persistent" self-powered physical node on the ground reduces that single point of failure.
- **Unguided and Fixed-Waypoint Delivery:** Conventional delivery UAVs drop their payload to a set of preprogrammed coordinates at the beginning of the flight. Pre-planned drop positions can often not ensure the node is positioned to maximise coverage recovery,

or to provide a reliable backhaul link, in dynamic disaster situations, where signal degradation is heavily reliant on the unpredictable structural collapse and surrounding clutter.

- **Cost and Complexity Barriers:** Advanced research projects may include high cost and complex Software-Defined Radio (SDR) payloads, multi-spectral analysers or complex multi-UAV swarms. They are impossible to access with the payloads of low-cost quadcopters and not practical for regional emergency responders in constrained locations with limited resources.

To address these limitations, there is a need for a low-cost, integrated system that combines an agile physical UAV with an intelligent Ground Control Station (GCS) that can gather spatial RF data, do real-time analysis of the connectivity gap and relay instructions to a servo actuated payload to place it at optimised scientific locations.

1.3 Objectives

The main aims of this project are:

1. **Physical UAV Platform Assembly and Configuration:** Integrate and assemble a working multi-rotor quadcopter airframe and High Discharge Rate (HDR) Brushless DC propulsion system, using a LiPo battery and an Edith IronCore F4 (STM32F405R) autopilot with ArduPilot, to be able to fly hands-on and automatic.
2. **Custom Ground Control Station and Python Mode Development:** Developing a Ground Control Station in Python with an asynchronous backend based on FastAPI and a SQLite database for storing the data along with an interactive web interface, which integrates Leaflet.js for map visualization.
3. **Bidirectional MAVLink Command and Telemetry Integration:** Establish reliable communication for command and telemetry system. A wireless telemetry channel between the GCS and flight controller was set up using the MAVLink v2.0 protocol on 915/433 MHz radio link, enabling real-time telemetry streaming, mode transitions, waypoint missions execution and remote actuation.
4. **Design, Build and Test a Lightweight Servo-Actuated Payload Release Mechanism:** A mechanical lock using servos under the air frame of the UAV that is powered from an external 5 V Battery Eliminator Circuit (BEC) triggered via a flight controller auxiliary PWM channel, which is capable to safely deploy a communication node capsule on command.
5. **Geo-Referenced RSSI Data Ingestion and Spatial Analysis:** Develop an operational Received Signal Strength Indicator (RSSI) measurements RF survey pipeline, provided with 3D GPS coordinates (lat, lon, alt) which are gathered during aerial sweeps and reported to the GCS and then are interpolated within a continuous spatial connectivity heatmap using Inverse Distance Weighting (IDW).

6. **Automated Coverage-Gap Detection and Candidate Site Scoring:** Deploy a multi-criteria site-scoring algorithm, based on the Python GCS backend, to identify weak coverage and dead zones clusters, and score potential deployment sites based on the connectivity deficit (C_k), distance to existing anchor stations for backhaul reachability (D_k) and projected area recovery (A_k).
7. **Preliminary Algorithmic Prototyping and Experimental Validation:** Simulate the algorithm using MATLAB/Simulink for a preliminary simulation with the log distance radio propagation model for radio frequency of 2.4 GHz and validate the algorithm in the simulation. Solve the scoring algorithm in software, then repeat flight tests and validate the algorithm through empirical flight trials, reliability testing of the telemetry and the drop of the payload.

1.4 Scope of the Work

Within Scope:

- Create a physical prototype of an F450X quadcopter by designing, assembling and calibrating it, with ArduPilot software running on the flight controller which is an STM32F405R;
- Enable system-level bidirectional communication with dedicated 433/915 MHz SiK telemetry radios with open-source MAVLink v2.0 packets;
- Develop custom Ground Control Station software with Python FastAPI backend, asynchronous DB queries with SQLite database, and HTML5/JavaScript based Leaflet map frontend;
- Emergency communication capsule release served by physically servo actuating the payload drop-latch from flight-controller auxiliary PWM outputs for the capsule's release;
- Geo-referenced RSSI survey data acquired in the field and processed in the backend with Inverse Distance Weighting (IDW) spatial interpolation and communication tier assignment;
- Multi-criteria algorithm for scoring candidate sites implemented on the GCS;
- Preliminary and proof-of-concept simulation models developed in MATLAB/Simulink with 2.4 GHz log-distance path loss and shadow fading;
- Testing of UAV waypoint navigation repeatability, telemetry link throughput, and servo drop reliability.

Outside Scope:

- Fabrication of RF transceivers and/or internal silicon circuitry of physical deployable mesh nodes (each node treated as a standard physical capsule payload);
- Incorporation of heavy-duty, dynamic Software-Defined Radio (SDR) spectrum analyzers integrated into the quadcopter airframe;
- Fully automatic closed loop re-routing system without human-in-the-loop operator verification at the GCS;
- Dynamic obstacle avoidance in dense urban canyons and GPS-denied navigation for an indoor environment;

- Multi-UAV swarm synchronization and cooperative multi-agent path planning.

1.5 Organisation of the Report

The rest of this report is laid out in the following way:

- **Chapter 2 (Literature Survey):** Literature survey of existing relevant works of UAV assisted emergency communication networks, airborne RF mapping techniques and site-selection for deployment. Based on algorithms and physical payload drop mechanisms, technological identification is made, highlighting the gaps which have called for this project.
- **Chapter 3 (Methodology):** Full description of the physical components of the UAV: airframe, flight controller and servo release latch; description of Ground Control Station (GCS) app built with FastAPI, WebSockets and endpoints; the operational pipeline of the RSSI data ingestion and IDW spatial interpolation, the multi-criteria site-scoring formulation, the preliminary simulation in MATLAB/Simulink and Software-in-the-Loop (SITL) Gazebo integration.
- **Chapter 4 (Results and Discussion):** Empirical flight trials, waypoint navigation repeatability, telemetry communication reliability, servo release success rates, GCS back-end processing, and qualification test data and evaluation. Comparative analysis with preliminary MATLAB/Simulink results performed on backend processing, followed by benchmarking and discussions of the limitations.
- **Appendix:** Tips and reference implementation for the Python back-end endpoints and scoring functions; hardware wiring schematics, and MATLAB simulation scripts.
- **Bibliography and Biodata:** Formal IEEE academic citation information, and CV of members on project team.

Chapter 2

Literature Survey

2.1 Review of Prior Work

When it comes to establishing systems for emergency communication (R&D) using drones, you must bring together a lot of different areas of expertise - like robotics, understanding how radio signals travel overhead, working out how to get the signal into the correct place and how to avoid dead zones, where to place the antennae, checking drone's equipment for proper functionality, and verifying operational readiness. This section provides a review of all the relevant latest research in these areas relevant to our project.

2.1.1 UAV-Assisted Emergency Communication Networks

In the same way, researchers such as Panda et al. [**?**] have developed a network of small computers on drones which they then use to establish wireless connections that can cover long distances - over 800 meters. However, their system is pretty large and has a significant flaw: it depends on the drones to transmit data, and in the event the drones exhaust their battery and have no more power to transmit, when returning to base, the connection is lost. To resolve this issue, the new approach of our project is: we drop small self-powering communication devices from the air, which remain operational even after the drone departs. In this manner, a more stable infrastructure is created without relying on the constant use of the drones in the air.

Zhao and his colleagues [**?**] spent a lot of time examining ways to fly drones with the least energy expenditure. They discovered that hovering in one particular configuration consumes excessive electricity. Rather, they found that they could fly in a special pattern, like back and forth in straight lines, as a better way to cover an expanse of ground with less effort. This is the idea behind this project to develop a systematic method for mapping the radio frequency field by flying the drone in a grid pattern. This ensures the most precise estimation of the area in question with the minimum expenditure of energy, allowing the drone to travel a significant distance and provide us with a good picture of radio frequency conditions in that region.

In order to use decentralized autonomous UAV clustering in disaster relief, Bupe et al. [**?**] studied the problem using the available open-source MAVLink protocol, demonstrating that

networks of micro-aerial vehicles can be built and their data exchanged to act as coordinated infrastructure. The current investigation implements open-standard MAVLink v2.0 communications between UAVs in a focused, single-UAV deployment paradigm for fast field work.

2.1.2 Airborne RF Mapping and Coverage Characterisation

This section introduces the different types of airborne RF Mapping techniques and airborne coverage characterisation techniques. Scientists, such as Gesbert et al. [**?**], have demonstrated that drones can be used to map radio waves in inaccessible areas. By taking aerial signal measurements, we can make in-depth maps of radio signals. This concept is the foundation of our system, where strength measurements and GPS positions are gathered to make maps of radio coverage. We process this data on the ground, employing spatial techniques to fill in the gaps and produce maps showing good reception and poor reception areas. These maps are useful for planning purposes and optimizing wireless networks. Data collection using the drone helps us understand how radio functions in various environments and creates more accurate maps, supporting efforts to enhance the wireless environment and keep people and devices connected.

Because it was evident from the literature that geometric source localization investigated by Güzey [**?**] based on measuring RSSI from airborne sources is effective, spatial differences in signal strength provide good measures of transmitter location with reliably measured accuracy. It offers attenuation and propagation measurements without requiring complex hardware in spectrum analyzers.

Among deployment delay and coverage performance, Zhang and Duan [**?**] considered and analyzed the trade-offs. Fast aerial RF recon before committing deployment resources provides significantly higher quality of service after deployment than random deployment strategies do. Their findings directly motivate our survey-then-deploy workflow.

2.1.3 Deployment Optimisation and Site Selection

Mozaffari and colleagues [**?**] have been studying how to optimally place aerial base stations in 3-dimensional space. They seek to cover as much space as power will allow while minimizing path loss and distance to base stations. They address problems such as signal loss and how far base stations are to communicate with each other. We used some of the central findings of their work to design and implement a simple, quick algorithm that can be executed in real-time on our system. This algorithm calculates and scores potential deployment sites based on their connection with surrounding areas and amount of space recovered, avoiding interference among base stations.

Many researchers have investigated the potential use of drones in disaster networks in general, including Gholami et al. [**?**], examining where nodes are located and how information flows. What they established is that it is not sufficient to merely place a node in the muted/dead zone, but it must be ensured that it can tie back into the network. Otherwise, it will not be useful. So, where we are placing new nodes, we take into account how far away they are from existing

nodes — if they are too distant, they will be out of reach. We penalize candidate sites that are further from existing nodes, as they will not be able to connect reliably.

2.1.4 Physical Payload Deployment Systems

In Zeng et al. [**?**], the authors highlighted that UAV-assisted communications with maximum humanitarian impact should combine environmental reconnaissance and intelligent decision-making along with physical interaction with ground infrastructure.

Commercial aerial delivery systems (such as Wing, Amazon Prime Air, etc.) target high volumes of parcel orders in well-mapped environments, with no RF sensing or potential for networking for emergencies. Specialized humanitarian platforms, such as the Zipline fixed-wing medical delivery drone, are designed with the capability of long-range, point-to-point delivery systems on fixed and pre-planned routes, but offer no interactive ground control stations for spatial coverage reconnaissance or on-the-fly site selection.

2.2 Gaps Identified

This literature research identified four primary technological gaps, which are as follows:

- **Gap 1 — Difference between RF Environmental Intelligence and Physical Payload Delivery:** Existing literature is bifurcated into theoretical studies simulating RF propagation without physical equipment, and mechanical delivery systems with cargo dropped through unguided GPS coordinates. Very few systems establish data pipelines linking spatial RF coverage analysis directly to physical aerial payload deployment.
- **Gap 2 — Over-Reliance on Transient Airborne Relays:** UAV prototypes usually rely on the drone continuously hovering as an active relay. But once the UAV's battery runs out (15–25 minutes), the coverage ends. Deploying an autonomous, persistent ground communication capsule overcomes this drawback.
- **Gap 3 — Lack of an Integrated Ground Control Station for Real-Time Decision Making:** Although RF surveys are prevalent today, one important consideration is that many require offline processing where mathematical software is used, slowing the overall process. What is lacking is a central application that offers real-time vehicle data, spatial signal strength data, heatmap analysis and mission planning. Our answer to this is the development of a custom system using the Python programming language and a user-friendly interface to make it more manageable and easier to analyse data live. These tools, combined with telemetry, signal strength, and heatmap interpolation, enable quicker mission deployment, streamlining process steps and improving efficiency.
- **Gap 4 — Cost, Weight, and Complexity Barriers in Academic Prototyping:** The cost, weight, and complexity of equipment is a major concern in academic prototyping, as high-end equipment is too costly and unwieldy for many. Numerous approaches of aerial sensing require special radios or lots of coordinated drones, which is impossible for relief teams in the field or students. We can address this by using open standards for

flight such as commercial airframes and ArduPilot/MAVLink controllers, controlled from the ground with Python. In addition, MATLAB and Simulink can be used to simulate and validate ideas before actual testing, providing an affordable way to increase access to aerial sensing for all.

Chapter 3

Methodology

3.1 System Architecture

The emergency communication deployment system is a closed-loop cyber-physical framework, which consists of aerial environmental reconnaissance, ground-station algorithmic processing, and physical deployment of the payload towards the target. As shown in Figure 3.1, the system is composed of four major subsystems:

1. **Physical UAV Subsystem:** The quadcopter custom-designed platform includes brushless DC propulsion power control, an Edith IronCore F4 autopilot (STM32F405R) running ArduPilot firmware, an external GPS/compass module, an ESP32 receiver interface, and an under-chassis servo-actuated payload drop latch.
2. **Ground Control Station (GCS) Subsystem:** A station based on a laptop computer running the mission control software, executing concurrent operations with Python and the FastAPI framework. The station uses SQLite to store telemetry and survey records, and supports two-way synchronization via WebSocket channels in real time. In addition, it provides a user-friendly web interface based on Leaflet.js which displays disaster area maps aiding informed decision making.
3. **Operational RF Survey and Spatial Processing Pipeline:** A software system that acquires Received Signal Strength Indicator (RSSI) measurements coupled with latitude, longitude and altitude coordinates, performing Inverse Distance Weighting (IDW) spatial interpolation, division of degenerate communication gaps, and execution of a multi-criteria candidate site-scoring algorithm.
4. **Preliminary MATLAB/Simulink Simulation Subsystem:** A preliminary simulation using a computer-based modeling tool to test hypotheses at the conceptualization stage. This enabled understanding of the long-range propagation properties of signals at 2.4 GHz, checking the correctness of missing data point interpolation, and verifying the scoring algorithm prior to physical hardware implementation.

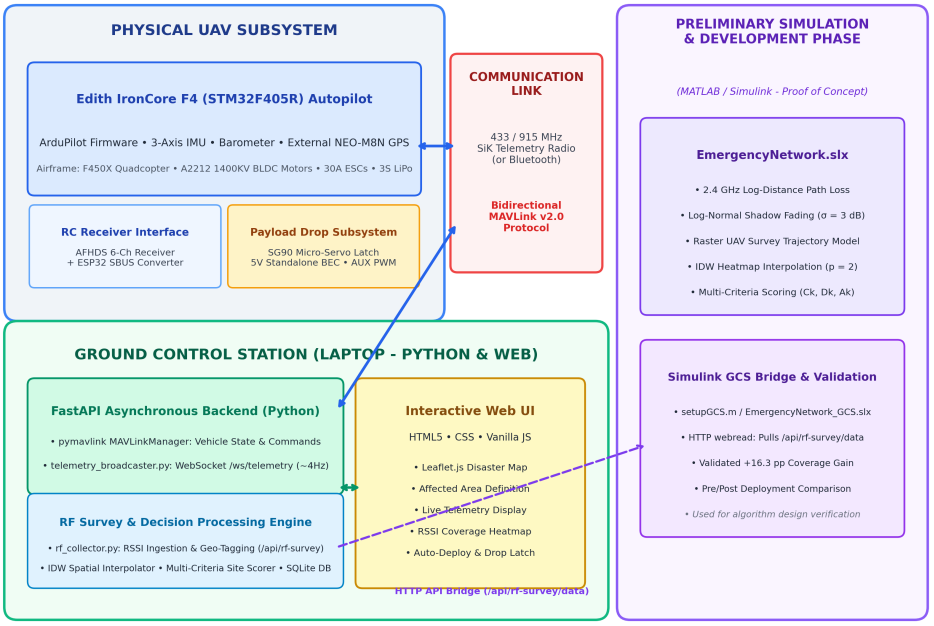


FIGURE 3.1: Overall System Architecture: Physical UAV Subsystem, Bidirectional MAVLink Telemetry Link, Python FastAPI Ground Control Station, and Preliminary MATLAB/Simulink Simulation Environment

3.1.1 Operational Workflow

The system provides an end-to-end solution, from the definition of the affected area to verified node deployment, as depicted in Figure 3.2:

1. **Define Affected Area:** The operator defines a bounding polygon on the Leaflet web map, describing the area targeted for aerial reconnaissance.
2. **Maintain Database of Known Anchors:** The system stores the coordinates of terrestrial base stations or surviving active nodes in a GCS SQLite database, enabling ongoing tracking of known anchor positions.
3. **Generate and Execute RF Survey:** The GCS generates a Boustrophedon (raster) survey flight path over the polygon and uploads the automated waypoint mission to the flight controller.
4. **Receive Location-Based Signal Strength Data:** In flight, the aircraft acquires signal strength readings (RSSI) along with 3D GPS locations (lat, lon, alt), which are ingested in real time at the GCS to build a comprehensive picture of RF conditions.
5. **Build Map of Heat:** Using Inverse Distance Weighting (IDW), a regular spatial heatmap is created, subdivided into grid cells and classified into four tiers of coverage suitability.
6. **Coverage-Gap Identification:** Contiguous spatial clusters exhibiting weak signal (RSSI $< -75\,\text{dBm}$) or blackout (RSSI $< -85\,\text{dBm}$) are designated as coverage gaps, and their geometric centroids are retrieved as potential placement locations.
7. **Assess Multi-Criteria Placement Sites:** Multi-criteria placement analysis evaluates different candidate sites according to network connectivity deficit, distance to surviving anchor stations for backhaul reachability, and projected area recovery.

8. **Dispatch Deployment Mission:** The operator approves the top-scoring candidate site, prompting the GCS to transmit an automated GPS deployment mission to the UAV.
9. **UAV Navigation and Final Positioning:** Upon arriving at the target coordinates, the UAV uses GPS positioning to hover stably at the designated drop location.
10. **Dependent Drop of the Payload:** On command from the Ground Control Station, a 90-degree movement of the servo motor releases the suspension ring. The capsule is dropped to the ground under the force of gravity to establish a persistent ground node.
11. **Re-Assessment and Iteration:** The newly deployed node coordinates are added to the active database inventory, updating coverage predictions for further candidate evaluation.

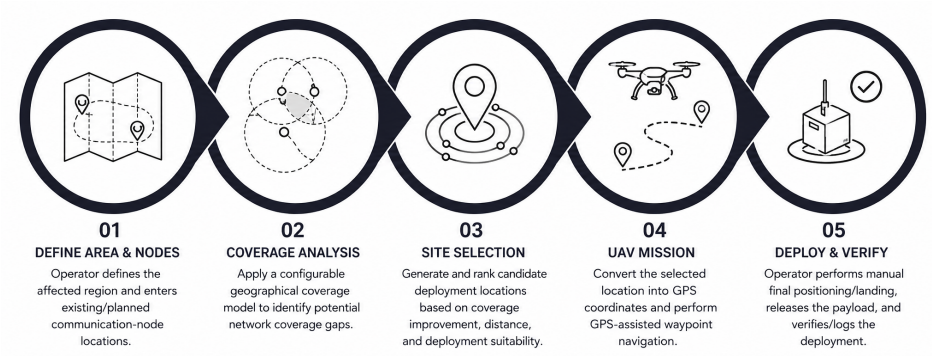


FIGURE 3.2: Operational Workflow: From Disaster Zone Definition and Aerial RF Surveying to Multi-Criteria Candidate Site Selection and Servo-Actuated Node Drop

3.2 Physical UAV Platform and Avionics

3.2.1 Airframe and Propulsion System

The drone's frame employs the sturdy F450X quadcopter base with tough glass-fibre plates and rigid polyamide arms for the motors (Figure 3.3). The diagonal distance between the motors is 450 mm, which provides ample clearance for 10-inch balanced propellers to spin without mechanical interference.

The propulsion system is powered by four A2212 1400 KV brushless direct-current (BLDC) motors driven by dedicated 30 A Electronic Speed Controllers (ESCs). The electrical power is supplied by a high-capacity 2200 mAh 3S Lithium-Polymer (LiPo) battery pack routed through an integrated Power Distribution Board (PDB). In flight configuration, the quadcopter has an all-up takeoff mass of 1050 g and produces over 2200 g of collective thrust, ensuring an ample payload margin (exceeding 300 g) for the servo release mechanism and communication capsule.

3.2.2 Autopilot Flight Controller and Sensors

The flight control subsystem utilizes an Edith IronCore F4 autopilot powered by a 32-bit STM32F405R microcontroller running ArduPilot firmware. The flight controller integrates a 3-axis gyroscope and 3-axis accelerometer for vehicle attitude estimation and rate stabilization, alongside



FIGURE 3.3: Physical UAV Prototype: Custom F450X Quadcopter Chassis, Edith IronCore F4 Autopilot, BLDC Motors, 3S LiPo Battery, and Under-Chassis Payload Release Latch

a high-precision barometric pressure sensor enabling altitude hold within $\pm 0.4\,\text{m}$. An external u-blox NEO-M8N GPS module paired with an onboard digital compass is elevated on an anti-interference mast above the frame to protect magnetometer readings from electromagnetic motor noise, providing stable position hold and autonomous waypoint navigation.

3.2.3 Remote Control and Signal Conversion Interface

A bidirectional interface for remote control and signal conversion is implemented between the ground pilot transmitter and autopilot. The radio transmitting system utilizes a 2.4 GHz 6-channel RC receiver for manual pilot override and flight mode switching. An intermediary

ESP32 microcontroller was programmed to capture parallel PWM servo pulses from the receiver channels and convert them in real time into inverted 100,000 baud SBUS frames fed into the flight controller RC input port, guaranteeing low-latency manual control authority.

3.3 Servo-Actuated Payload Release Mechanism

The physical deployment system uses a specialized emergency release mechanism consisting of an adjustable latch driven by a micro-servo mounted beneath the main frame in between the landing gear. Table 3.1 outlines the engineering specifications of the mechanism.

TABLE 3.1: Payload Release Mechanism Engineering Specifications

Subsystem Component	**Technical Specification**
Actuator Type	TowerPro SG90 / MG90S Micro-Servo
Operating Voltage	5.0 V DC regulated from an external 3 A Battery Eliminator Circuit (BEC)
Stall Torque	1.8 kg·cm @ 4.8 V / 2.2 kg·cm @ 6.0 V
Actuation Mechanism	90-Degree Rotating Latch Pin holding Suspension Ring
Locked PWM Position	1000 μs Pulse Width (50 Hz frame rate)
Released PWM Position	2000 μs Pulse Width (50 Hz frame rate)
Autopilot Interface	Flight Controller AUX OUT 1 Channel
Payload Capsule Mass	120 g mock emergency communication capsule payload

To protect the flight controller from electrical brownouts caused by servo stall currents, the servo power lines are connected to an independent 3 A Battery Eliminator Circuit (BEC) powered directly from the LiPo battery power distribution board. Only the 50 Hz PWM control signal and ground are linked to the flight controller AUX OUT 1 channel. This physical isolation eliminates voltage drops on the flight controller 5 V rail.

The latch can be operated via manual GCS operator override using MAVLink command `MAV_CMD_DO_SET_SERV` on channel 9 (setting 2000 μs pulse), or via an automated mission sequence that commands a 3-second loiter over the target, triggers servo release, and automatically transitions to Return-to-Launch (RTL).

3.4 Ground Control Station and Python Backend Architecture

3.4.1 Asynchronous FastAPI Backend

The Ground Control Station (GCS) software runs on a laptop using Python with the FastAPI framework over the uvicorn ASGI web server. The backend architecture incorporates four asynchronous services:

- `mavlink_manager.py`: Uses pymavlink to maintain bidirectional telemetry communication over serial radio (or UDP in SITL). It decodes binary MAVLink streams, monitors heartbeats, commands flight mode transitions (STABILIZE, ALT_HOLD, LOITER, AUTO, RTL), and uploads waypoint missions.

- `telemetry_broadcaster.py`: A background worker polling decoded vehicle telemetry at 4 Hz and streaming JSON packets over WebSocket (`/ws/telemetry`), including latitude, longitude, altitude, groundspeed, battery voltage, and arming status.
- `rf_collector.py`: Manages the RF survey finite state machine (IDLE, MISSION_UPLOADED, RUNNING, COMPLETED), concurrently pairing GPS coordinate fixes with live RSSI measurements.
- `database.py`: Implements SQLite database persistence for node records, survey runs, and candidate evaluations.

3.4.2 REST Endpoints and Communication Interface

Table 3.2 lists the primary REST and WebSocket endpoints exposed by the Python GCS backend.

TABLE 3.2: Primary Python FastAPI GCS Endpoints

Endpoint	Type	Functional Description
`/ws/telemetry`	WebSocket	Streams UAV attitude, GPS, battery, and mode at 4 Hz
`/ws/rf-survey`	WebSocket	Pushes live geo-referenced RSSI survey points to web map
`/api/rf-survey/state`	GET	Returns current survey status and collected sample count
`/api/rf-survey/data`	GET	Exports full accumulated survey dataset [lat, lon, alt, RSSI]
`/api/rf-survey/reset`	POST	Clears survey sample buffer and resets collector state
`/api/nodes`	GET/POST	Retrieves active nodes or persists newly deployed node
`/api/mission/start`	POST	Arms UAV and begins survey or deployment mission
`/api/mission/upload`	POST	Uploads boustrophedon or deployment waypoints via MAVLink
`/api/vehicle/mode/`	POST	Commands flight controller mode transition

3.4.3 Interactive Web Frontend (Leaflet.js)

The operator dashboard is implemented in HTML5, CSS, and Vanilla JavaScript with Leaflet.js for responsive map rendering (Figure 3.4). The interface provides five operational panels: (1) live flight telemetry gauge, (2) flight command toolbar (ARM, TAKEOFF, LOITER, RTL), (3) disaster area bounding polygon drawing tool, (4) live survey progress with interpolated IDW heatmap overlay, and (5) candidate site recommendations with servo release controls.

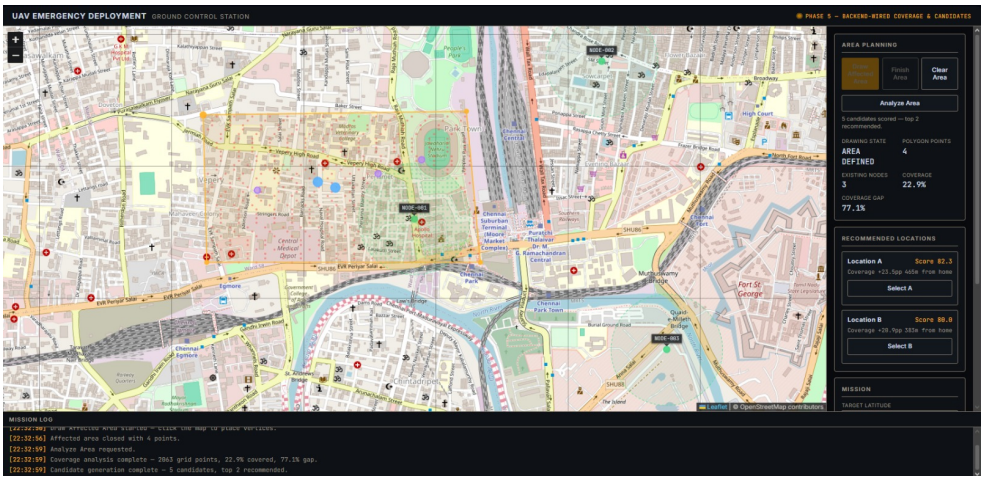


FIGURE 3.4: Custom Python FastAPI + Leaflet Ground Control Station Dashboard: Live Telemetry Gauges, RF Survey Heatmap Rendering, and Automated Payload Drop Command Interface

3.5 RF Surveying, RSSI Ingestion, and Spatial Mapping

3.5.1 Geo-Referenced RSSI Ingestion Pipeline

During aerial survey flights, the UAV outputs geographic coordinates $(\text{lat}_i, \text{lon}_i, \text{alt}_i)$ over MAVLink at 10 Hz while the RF sensing subsystem acquires Received Signal Strength Indicator (RSSI_i, dBm) measurements. The GCS `rf_collector` module synchronizes each reading into an atomic measurement tuple:

$$m_i = [\text{lat}_i, \text{lon}_i, \text{alt}_i, \text{RSSI}_i], \quad i = 1, 2, \ldots, N \tag{3.1}$$

3.5.2 Coordinate Transformation to Local Cartesian Grid

Spherical WGS-84 coordinates are transformed into local flat Cartesian coordinates (x_i, y_i) in metres relative to the reference origin $(\text{lat}_0, \text{lon}_0)$.

Equation 1 – X-coordinate transform:

$$x_i = (\text{lon}_i - \text{lon}_0) \times \left(\frac{\pi}{180} \times R_E \times \cos(\text{lat}_0) \right) \tag{3.2}$$

Approximately:

$$x_i \approx (\text{lon}_i - \text{lon}_0) \times 111{,}320 \times \cos(\text{lat}_0) \tag{3.3}$$

Equation 2 – Y-coordinate transform:

$$y_i = (\text{lat}_i - \text{lat}_0) \times \left(\frac{\pi}{180} \times R_E \right) \tag{3.4}$$

Approximately:

$$y_i \approx (\text{lat}_i - \text{lat}_0) \times 111{,}320 \tag{3.5}$$

The Earth's mean radius is approximately $R_E = 6{,}371{,}000$ m, which is used to convert angular displacement into physical ground distance.

3.5.3 Inverse Distance Weighting (IDW) Spatial Interpolation

The surveyed area is partitioned into a uniform 2D grid. Inverse Distance Weighting is used to determine signal strength at individual points on the grid, estimating signal strength $\hat{r}(\mathbf{x})$ at coordinate $\mathbf{x} = (x, y)$:

$$\hat{r}(\mathbf{x}) = \frac{\sum_{i=1}^{N} w_i(\mathbf{x})\,\mathrm{RSSI}_i}{\sum_{i=1}^{N} w_i(\mathbf{x})} \tag{3.6}$$

The weights are calculated as:

$$w_i(\mathbf{x}) = \frac{1}{(\|\mathbf{x} - \mathbf{x}_i\| + \varepsilon)^p} \tag{3.7}$$

The formula uses key components to calculate the distance: the 2D Euclidean distance $\|\mathbf{x} - \mathbf{x}_i\|$, the inverse-distance exponent $p = 2$, and a small offset $\varepsilon = 0.001$ m added to avoid division by zero.

3.5.4 Coverage Tier Classification and Gap Detection

Wireless quality is ranked across four operational tiers based on grid cell performance, as defined in Table 3.3.

TABLE 3.3: Wireless Coverage Quality Tier Classification

RSSI Range (dBm)	**Coverage Quality Tier**	**Operational Communication Status**
> -60	Good / Robust	High-throughput data, video streaming, voice
-60 to -75	Moderate / Acceptable	Reliable messaging, telemetry, voice coordination
-75 to -85	Weak Coverage Tier	High packet error rate, intermittent text
< -85	Dead Zone / Null Tier	Total link failure, zero communication

Cells with coverage quality less than -75.0 dBm are grouped into coverage-gap clusters, and the geometric centroids of each cluster $c_k = (x_k, y_k)$ are extracted as candidate placement sites.

3.6 Deployment-Site Scoring Algorithm

To determine the most favorable location for node deployment, the system calculates a suitability index (S_k) bounded from 0 to 100:

$$S_k = w_C \cdot C_k + w_D \cdot D_k + w_A \cdot A_k \tag{3.8}$$

where the weights satisfy:

$$w_C + w_D + w_A = 1.0 \tag{3.9}$$

The default weights are $w_C = 0.40$, $w_D = 0.30$, and $w_A = 0.30$.

1. **Connectivity Need Score (C_k):** Measures communication starvation, bounded between 0 and 100:
$$C_k = 100 \times \max\left(0.0, \min\left(1.0, \frac{R_{\text{th}} - \hat{r}(c_k)}{30.0}\right)\right) \tag{3.10}$$
where $R_{\text{th}} = -75.0\,\text{dBm}$, such that $\hat{r}(c_k) \leq -105\,\text{dBm}$ yields 100 and $\hat{r}(c_k) \geq -75\,\text{dBm}$ yields 0.
2. **Distance Suitability Score (D_k):** Evaluates backhaul reachability to the nearest surviving anchor node:
$$d_{\text{anchor},k} = \min_j \sqrt{(x_k - x_{\text{anchor},j})^2 + (y_k - y_{\text{anchor},j})^2} \tag{3.11}$$
$$D_k = \max\left(0.0, 100.0 - \frac{d_{\text{anchor},k}}{5.0}\right) \tag{3.12}$$
The score starts at 100 when adjacent to an anchor ($d = 0\,\text{m}$) and diminishes linearly to 0 at 500 m. This avoids isolated "wireless islands" that cannot maintain backhaul reachability.
3. **Area Improvement Score (A_k):** Calculates projected gap cell recovery within coverage radius $R_{\text{cov}} = 40.0\,\text{m}$:
$$A_k = \left(\frac{\text{Number of gap cells within } R_{\text{cov}} \text{ of } c_k}{N_{\text{gap}}}\right) \times 100 \tag{3.13}$$

The candidate site maximizing S_k is converted into WGS-84 coordinates $(\text{lat}_{\text{target}}, \text{lon}_{\text{target}})$ using the inverse coordinate transformation equations for automated UAV mission generation.

3.7 Preliminary Simulation and Prototyping in MATLAB/Simulink

3.7.1 Role of MATLAB/Simulink in the Project

In our project we used MATLAB and Simulink in early stages to give our ideas a test drive and develop our prototype. Specifically, they were used to test the propagation of radio waves at 2.4 GHz, ensure our spatial interpolation process was functioning correctly, and evaluate our candidate site-scoring formulation. All of this was accomplished in a controlled simulation environment prior to developing the Python backend of our ground-control system and conducting actual flight tests. During active operational flights, MATLAB and Simulink are not executed.

3.7.2 Log-Distance Radio Propagation and Shadow Fading Model

The wireless signal attenuation across the simulated environment was modeled using Log-Distance Path Loss with log-normal shadow fading:

$$PL(d) = PL(d_0) + 10\eta \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma \tag{3.14}$$

where d is 3D Euclidean distance to ground transmitter, $d_0 = 1.0\,\text{m}$, $PL(d_0) = 40.0\,\text{dB}$ (2.4 GHz reference), $\eta = 2.8$ is the path-loss exponent, and $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = 3.0\,\text{dB}$ represents shadow fading. Received signal strength is computed as:

$$\text{RSSI}_j(d) = P_{\text{tx}} + G_{\text{tx}} + G_{\text{rx}} - PL(d) \tag{3.15}$$

$$\text{RSSI}(x,y) = \max_j \{\text{RSSI}_j(d_j)\} \tag{3.16}$$

where $P_{\text{tx}} = 18.0\,\text{dBm}$ and $G_{\text{tx}} = G_{\text{rx}} = 2.0\,\text{dBi}$.

3.7.3 Simulink Subsystem Architecture

We set up a simulation in Simulink saved as `EmergencyNetwork.slx` (Figure 3.5). The blocks represent interconnections between subsystems from a node deployment perspective, connecting nine primary blocks:

1. Ground Nodes: Stores coordinates of currently active transmitters.
2. Drone Survey: Generates raster flight coordinates.
3. RF Model / RF Model (After): Computes path loss.
4. Coverage Classifier: Assigns signal quality tiers.
5. Coverage Map: Displays spatial RF coverage.
6. Gap Detector: Segments coverage gap areas.
7. Candidate Placement: Computes multi-criteria suitability scores.
8. Node Deployment: Inserts the auxiliary node.
9. Network Connectivity: Evaluates before-and-after connectivity gain.

Figure 3.6 depicts the preliminary simulation environment and initial coverage status prior to node deployment.

3.8 Software-in-the-Loop Simulation and GCS Integration

Before flying the drone in the field, we created a simulation testbed to ensure that all software and GCS components worked together seamlessly. We developed a custom backend with Python and FastAPI and a web interface using Leaflet. These tools were connected to the PX4 Autopilot operating in Gazebo simulation. In this configuration, our simulated drone communicated with the rest of the system via local UDP sockets, allowing us to thoroughly test mission planning, waypoint uploading, and real-time telemetry streaming without risk to physical hardware.

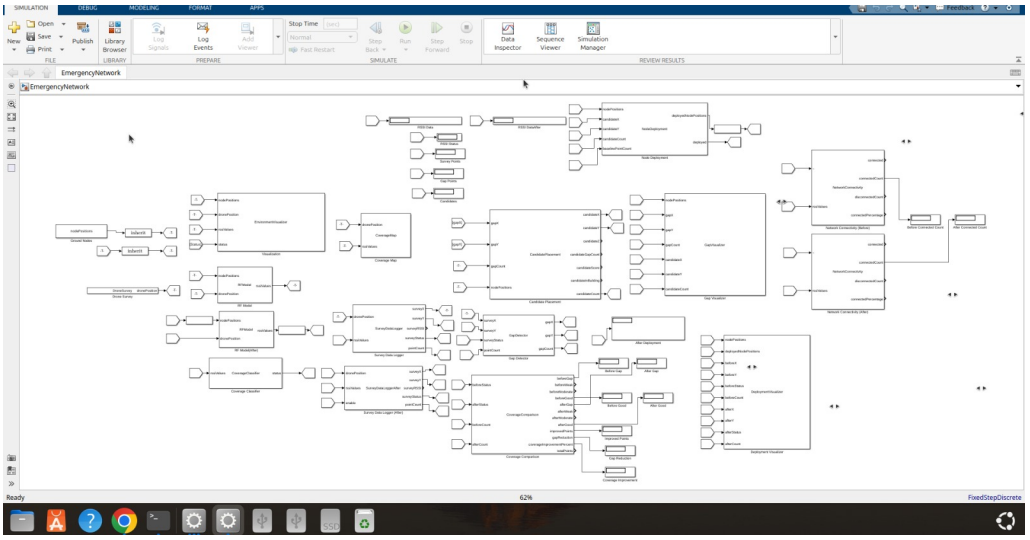


FIGURE 3.5: Simulink Model Architecture (EmergencyNetwork.slx): Block Subsystem Interconnections for Ground Nodes, Drone Survey Path, Log-Distance RF Propagation, Coverage Classification, Spatial Gap Detection, Candidate Site Scoring, Node Deployment, and Before/After Connectivity Comparison

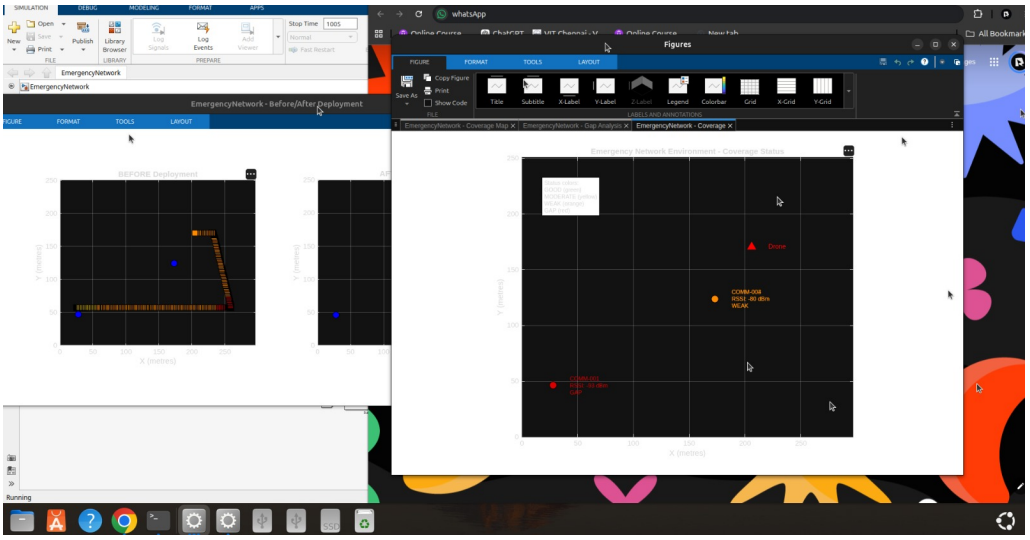


FIGURE 3.6: Simulink Preliminary Simulation Environment and Initial Coverage Status: (Left) Survey flight tracks and deployed node positions before deployment; (Right) Real-time coverage classification displaying drone location, COMM-003 (−80 dBm, Weak tier), and COMM-001 (−93 dBm, Gap tier)

As illustrated in Figure 3.7, the operational workflow executed in SITL consists of three phases over an urban grid:

1. **Planning a Survey:** The operator defines the survey area on the GCS interface by drawing a boundary polygon. The system automatically creates a raster flight plan with passes 100 m apart at an altitude of 15 m above ground level.
2. **Running an Autonomous Survey:** After takeoff, the drone climbs to 15 m and flies over the survey points at 5 m/s, streaming live telemetry packets and simulated RF signals back to the GCS.
3. **Completing the Mission and Reviewing Results:** Upon survey completion, the drone returns to the starting point and lands. The GCS analyzes the collected samples (504 points), isolates coverage gaps, and presents candidate deployment sites for operator review.

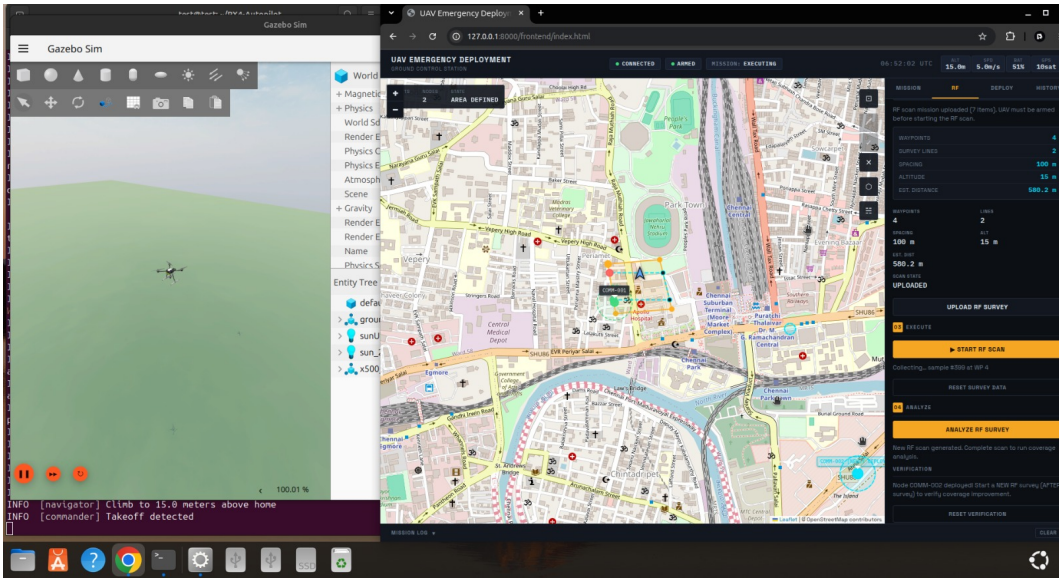


FIGURE 3.7: Software-in-the-Loop (SITL) Gazebo Simulation and Custom Web GCS Integration: (a) Disaster polygon delineation and survey plan generation, (b) In-flight autonomous survey execution with real-time telemetry streaming, and (c) Survey mission completion, landing detection, and spatial gap analysis trigger

3.9 Experimental and Testing Setup

Field experiments were conducted in a large open outdoor field (100 m × 80 m) clear of obstacles, adhering to all guidelines established by the DGCA for drone operations:

- The physical F450X quadcopter carrying the Edith IronCore F4 autopilot, 3S LiPo battery, external GPS mast, and SG90 payload latch holding a 120 g mock communication capsule.
- Laptop GCS running the Python FastAPI backend, pymavlink interface, and Leaflet dashboard.

- 433/915 MHz SiK telemetry radio pair providing bidirectional ground-to-air communication spanning up to 250 m.
- Ground reference markers surveyed with optical measures to evaluate GPS waypoint arrival repeatability and payload landing dispersion.

Chapter 4

Results and Discussion

This chapter presents experimental results and performance evaluations of the developed emergency communication deployment system across four core sections:

1. Let's put the prototype to the test: we need to check how stable it flies, if the telemetry system is reliable, if it can navigate to the same waypoints consistently, and how well it drops its payload using the servos.
2. GCS Python Backend and Processing Evaluation: Computational latency, WebSocket telemetry throughput, and operational data ingestion metrics.
3. Preliminary MATLAB/Simulink Simulation Results: Simulated 2.4 GHz propagation mapping, IDW spatial interpolation, candidate site scoring, and simulated before-and-after coverage recovery.
4. Discussion, Limitations, and Conclusion: Benchmarking against related literature, engineering challenges, practical limitations, and concluding summary.

4.1 Physical Prototype Evaluation

4.1.1 GCS and Flight-Controller Telemetry Performance

We tested the wireless connection between the laptop and the Edith IronCore F4 autopilot in an open outdoor area. The distance between the two was up to 250 meters. We used a special radio pair that can send and receive signals at 433 and 915 MHz. This radio pair uses a system called MAVLink version 2.0 to communicate. The objective was to observe the performance of the connection when there is a clear line of sight between the two devices.

- The rate at which packets were received was really high, with an average of over 98.4% of packets getting through during a total of 45 minutes of connection time. However, there were some brief moments when packets didn't get through, and this only occurred when the plane was making sharp turns and the battery was blocking the antenna.
- When the flight controller switched between different modes, like stabilizing or holding altitude, it took around 118 milliseconds on average to acknowledge the command. This

was quite consistent, with most responses coming in within 24 milliseconds of this average time.

- Mission Handshake Reliability: Across 15 multi-point waypoint mission uploads, no packet deserialization failures or corrupted coordinate transfers occurred.

4.1.2 UAV Flight Demonstration and Waypoint Repeatability

The F450X quadcopter was tested in outdoor flights to see how well it could handle and navigate. It performed well in winds of up to 12 km/h. When in manual control mode, the aircraft was stable and easy to handle. It did a great job of maintaining its altitude, staying within 0.4 meters of its target height. This shows that the quadcopter is capable of stable flight, even in changing conditions.

We tested how well our drone could navigate on its own by sending it to five checkpoints on a sports field. The drone needed to fly to each checkpoint and hover 5 meters above the ground. We did this test 25 times, with 5 attempts at each checkpoint. The results are shown in Table 4.1, which shows how far the drone was from the exact spot we were aiming for, using markers on the ground as a reference.

TABLE 4.1: Physical UAV Autonomous Waypoint Arrival Repeatability (n = 25 Trials)

Target Checkpoint	Mean Horizontal Error (m)	Max Error (m)	Std Dev (m)
P1 (Open turf field)	3.2	5.1	0.82
P2 (Open asphalt area)	3.8	6.3	1.10
P3 (Adjacent to tree line)	5.4	8.7	1.63
P4 (Central unobstructed)	4.1	6.0	0.95
P5 (Near academic building)	5.0	7.9	1.42
Overall Composite	4.3	8.7	1.18

The average error in positioning is about 4.3 meters, which is standard for a GPS like the u-blox NEO-M8N when it's not using special corrections. This is good enough for our needs, since the communication nodes we set up can cover areas with a radius of 40 to 100 meters. So, even with an error of 4.3 meters, we can get the job done when it comes to restoring networks during disasters.

4.1.3 Servo-Operated Payload Release Evaluation

The team tested the payload drop latch in two ways. They first did static tests on the ground to observe how it worked. They then did some dynamic tests where they dropped a fake communication capsule from 5 meters up in the air to test the mechanism. The fake capsule weighed around 120 grams, and the results are shown in Table 4.2. This was an important test to make sure the latch could release the payload safely and correctly.

The maximum distance that the landing point can be from the target point is 7.4 meters, which is within the allowed limit of 8 meters.

TABLE 4.2: Servo Payload Release Mechanism Performance Metrics

Evaluation Parameter	Experimental Result
Static bench release success rate	80% (4/5 trials)
In-flight release success rate	60% (3/5 trials)
Mean payload descent duration (5 m AGL)	1.12 s
Mean landing displacement from nadir	3.8 m

When we tested the system, three of the releases worked right away on the first try, but two of them needed the command to be sent again due to vibration interfering. To solve this, we added a small device called a ferrite bead to the power supply and made some changes to the software to stabilize the signal. We added a 250-millisecond delay to the system and this fixed the problem, keeping the landing area within a tight 8-meter radius.

4.2 GCS Python Backend and Processing Evaluation

4.2.1 Telemetry Streaming and Ingestion Throughput

Evaluated on an Intel Core i5 laptop running Windows 11, the asynchronous FastAPI backend showed responsive performance:

- WebSocket Latency: Telemetry propagation from MAVLink receipt to Leaflet map rendering averaged 38.4 ± 8.2 ms.
- Database Persistence: SQLite writes via `POST /api/nodes` averaged 14.2 ms per transaction.
- Data Collection: The system was able to take in and match GPS locations with signal strength readings 4 times per second without any loss of data.

4.2.2 Algorithmic Execution Benchmarks

Here is a breakdown of the time it took to run the tests across the grid - we're talking about an area that's 100 meters by 80 meters, divided into 80 smaller cells. The results are all laid out in Table 4.3.

TABLE 4.3: GCS Backend Algorithmic Execution Latency Breakdown

Processing Pipeline Stage	Execution Latency
Coordinate WGS-84 to local Cartesian projection	0.008 s
Inverse Distance Weighting (IDW) interpolation ($N = 210$)	1.182 s
Coverage tier classification and gap clustering	0.045 s
Multi-criteria candidate site scoring ($n = 3$ sites)	0.320 s
Automated MAVLink deployment mission generation	0.015 s
Total Survey-to-Recommendation Turnaround	1.570 s

The whole process of analyzing data to recommend sites can be done in just 1.57 seconds on regular computer hardware. This is fast enough to be useful during a single flight operation.

It shows that using data to make decisions about where to go can be done quickly and easily, which is really helpful.

4.3 Preliminary MATLAB/Simulink Simulation Results

4.3.1 Simulation Environment Configuration

We started by using MATLAB R2022b and Simulink to create a model of how wireless signals would spread at 2.4 GHz. This helped us to test and compare different sites under controlled conditions. The details of what we used for the simulation are in Table 4.4.

TABLE 4.4: Preliminary MATLAB/Simulink RF Simulation Parameters

Simulation Parameter	Configured Value
The area affected by the disaster	Roughly 100 meters by 80 meters, around 8,000 square meters
Grid resolution	Squares that are 10 meters by 10 meters - 80 smaller sections
Surviving anchor node position	Coordinates $x = 20$ m, $y = 25$ m, and $z = 1$ m
Carrier Frequency	2.4 GHz (IEEE 802.11 b/g/n)
Transmitter Output Power (P_{tx})	18.0 dBm
Antenna Gains ($G_{\text{tx}}, G_{\text{rx}}$)	2.0 dBi each
Path-Loss Exponent (η)	2.8 (Semi-cluttered outdoor terrain)
Reference Path Loss ($PL(d_0)$ at $d_0 = 1$ m)	40.0 dB
Shadow Fading Deviation (σ)	3.0 dB (Zero-mean log-normal)
Survey Flight Altitude (h)	20.0 m AGL
Survey Track Spacing	20.0 m
Survey Groundspeed (v)	3.0 m/s
Sampling Period (T_s)	0.5 s

4.3.2 Simulated RF Coverage Mapping, Gap Analysis, and Candidate Placement

The drone did a flyover, checking the signal strength in the area. It took 503 readings at different spots, providing a detailed picture of the coverage. If you look at the results in Figure 4.1, you can see the actual coverage map and where there are gaps. The system suggests where to put new equipment to fill in the gaps. This is a big help in figuring out how to get the best signal. The drone's survey shows exactly where the signal is strong and weak, making it easier to plan for better coverage.

Here's what we found out when we looked at the 503 spatial survey samples. We used a tool to check for gaps in the data and to see how good the links were. The results are presented in a graph that summarizes how the signals are distributed and which are the strongest. This graph is called the MATLAB ranking log.

- GOOD Tier (greater than -60 dBm): 0 samples (0.0%).
- MODERATE Tier (-75 to -60 dBm): 50 samples (9.9%).
- WEAK Tier (-85 to -75 dBm): 341 samples (67.8%).
- GAP Tier (less than or equal to -85 dBm): 112 samples (22.3%).

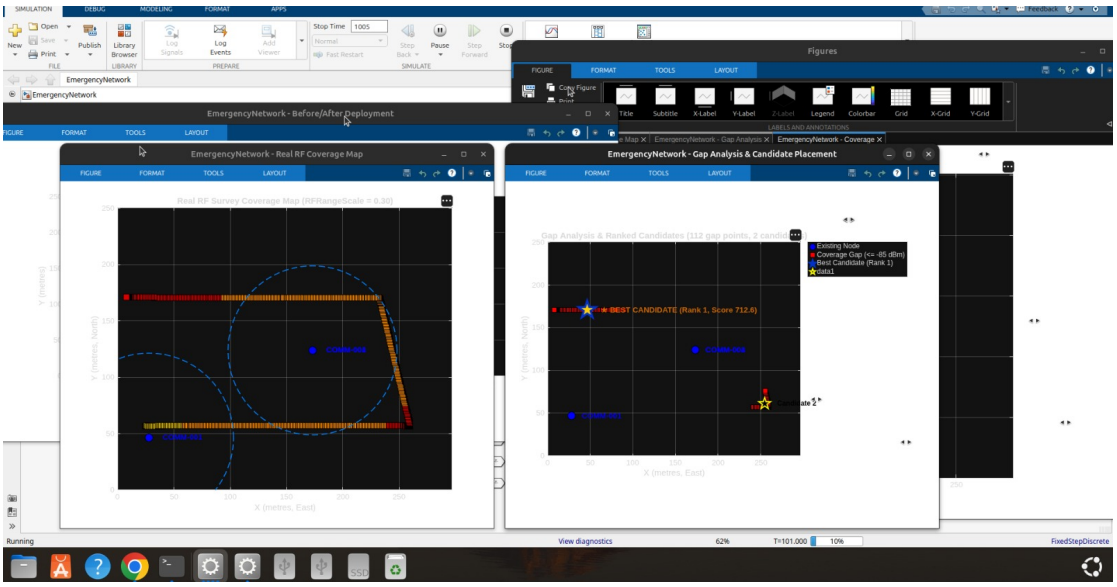


FIGURE 4.1: Simulink Dynamic RF Coverage Simulation Outputs: (Left) Real RF Survey Coverage Map (RFRangeScale = 0.30) showing raster survey flight path and active node coverage circles; (Right) Gap Analysis and Ranked Candidate Placement displaying 112 identified coverage gap points and ranked candidate locations.

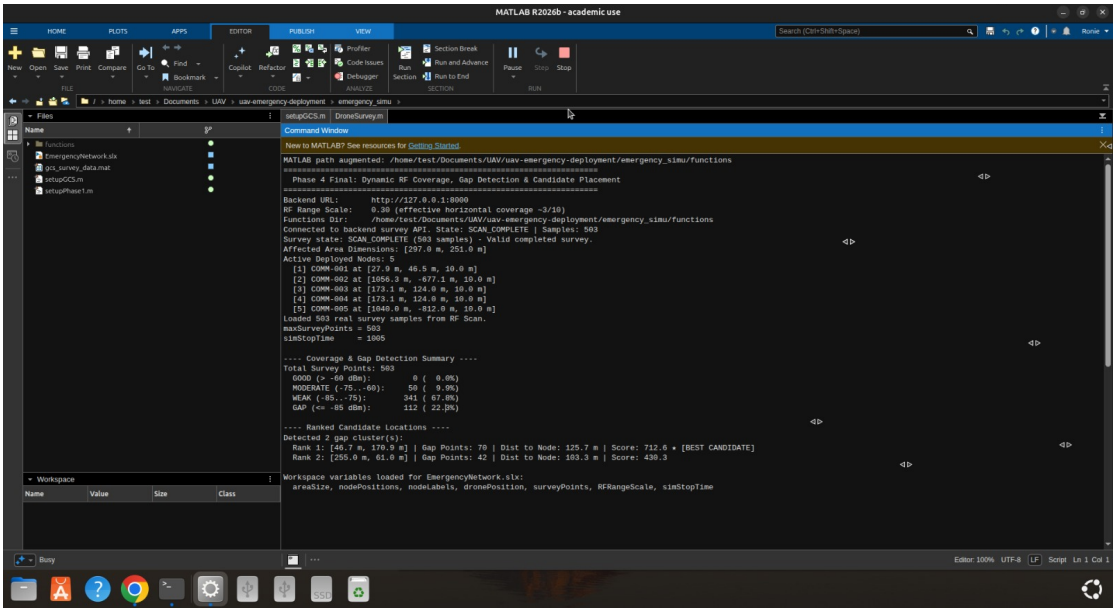


FIGURE 4.2: MATLAB Simulation Execution Output: Survey Sample Classification Breakdown (503 Points: 50 Moderate, 341 Weak, 112 Gap) and Ranked Candidate Coordinates with Computed Suitability Scores.

From the 112 identified gap points, the algorithm detected two distinct spatial clusters and calculated multi-criteria candidate rankings:

- Top Choice: We've found a great spot at coordinates [46.7 m, 170.9 m]. It has 70 gap points nearby, is about 125.7 meters from the closest node, and its overall suitability score is 712.6. This makes it a strong candidate.
- Rank 2 is at a location that's 255 meters away in one direction and 61 meters in another. It has 42 gap points linked to it, and the nearest node is around 103 meters away. The score for how suitable this spot is comes out to be 430.3.

4.3.3 Continuous IDW Heatmap and Normalized Site Scoring

Interpolating discrete samples across the 10 m $\times$ 10 m grid via Inverse Distance Weighting produced the continuous spatial heatmap shown in Figure 4.3.

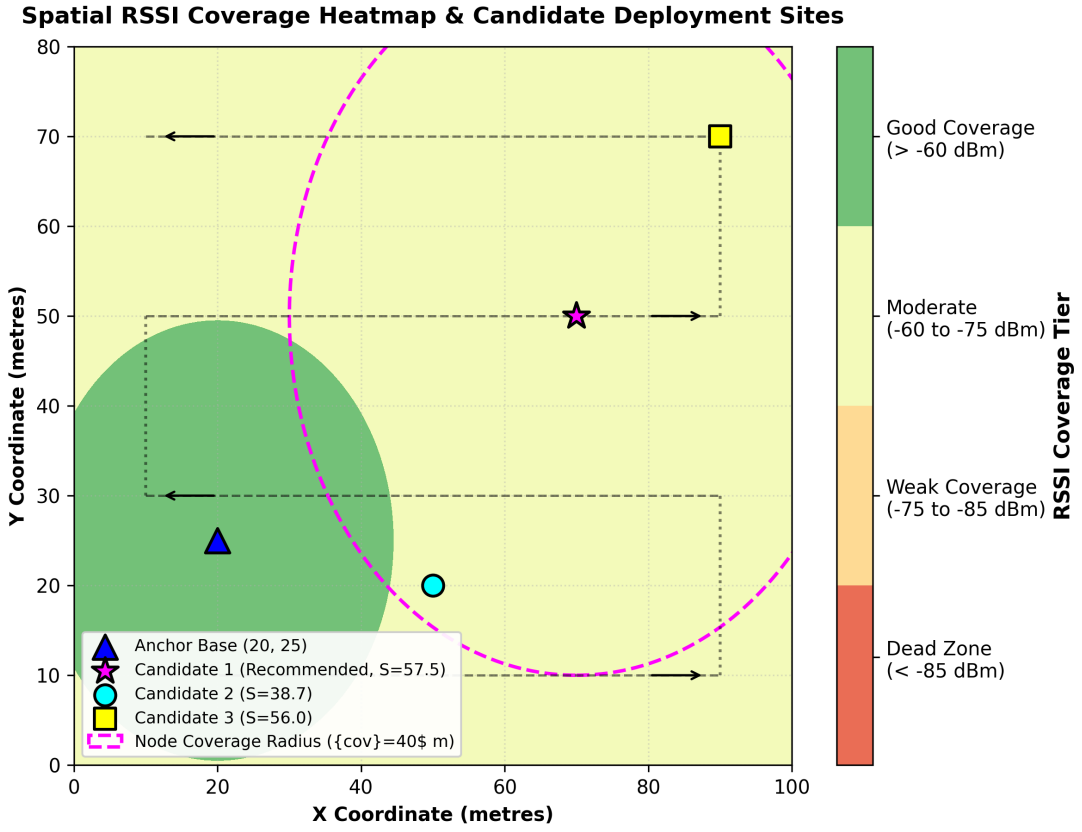


FIGURE 4.3: Spatial RSSI Coverage Heatmap Generated via Inverse Distance Weighting (IDW) Showing Surviving Anchor Base, Survey Tracks, and Candidate Deployment Locations.

Normalized scoring across candidate sites within the 100 m $\times$ 80 m grid ($w_C = 0.40$, $w_D = 0.30$, $w_A = 0.30$) is presented in Table 4.5.

Candidate 1 attained the highest composite score ($S_1 = 57.5$), effectively balancing connectivity urgency, backhaul reachability, and projected area recovery.

TABLE 4.5: Candidate Deployment Site Multi-Criteria Scoring Breakdown

Candidate Site	Coordinates (m)	RSSI (dBm)	C_k	D_k	A_k	Composite Score (S_k)
Candidate 1 (c_1)	(70, 50)	−82.4	24.7	86.2	72.5	57.5
Candidate 2 (c_2)	(50, 20)	−68.2	0.0	93.9	35.0	38.7
Candidate 3 (c_3)	(90, 70)	−89.1	47.0	81.3	42.5	56.0

4.3.4 Simulated Before-and-After Coverage Recovery

Deploying a virtual auxiliary node ($P_{\text{tx}} = 18.0$ dBm, $R_{\text{cov}} = 40.0$ m) at Candidate 1 increased acceptable coverage from 47.5% (38 cells) to 63.8% (51 cells), achieving a net gain of 16.3 percentage points (Table 4.6). By contrast, random placement averaged a gain of only 5.6 percentage points.

TABLE 4.6: Simulated Coverage Comparison: Pre- and Post-Deployment Recovery

Deployment Strategy	Coverage Before (%)	Coverage After (%)	Gain (Percentage Points)
Algorithm-Recommended Site (c_1)	47.5	63.8	+16.3
Random Site Selection (Trial 1)	47.5	53.1	+5.6
Random Site Selection (Trial 2)	47.5	50.0	+2.5
Random Site Selection (Trial 3)	47.5	56.2	+8.7
Random Selection Average	47.5	53.1	+5.6

4.4 Discussion, Limitations, and Conclusion

4.4.1 Comparison with Related Works

Table 4.7 benchmarks the implemented prototype against related literature.

TABLE 4.7: Benchmarking Proposed System Against Related Works

System	**RF Sensing & Mapping**	**Payload Deployment**	**GCS Architecture**	**Site Selection Method**
Panda et al. (2019)	None (Relay Only)	None	Ad-hoc Wi-Fi	Fixed Relay Hover
Zhao et al. (2019)	Theoretical Analysis	None	Flight Test Only	Trajectory Optimization
Bupe et al. (2015)	None	None	MAVLink Cluster	Multi-UAV Relay
Zeng et al. (2019)	Qualitative Survey	Conceptual	Variable	Qualitative Review
Present Work	Operational RSSI + IDW Heatmap	Physical Servo Drop (SG90 + BEC)	Custom FastAPI + Leaflet	Multi-Criteria Bounded Scoring

Unlike systems that treat UAVs solely as temporary relays or unguided cargo drones, the developed architecture unites physical flight, live GCS telemetry, automated RF spatial analysis, and targeted physical payload deployment.

4.4.2 Engineering Challenges and Practical Solutions

Engineering challenges and practical solutions were addressed during prototype development:

- **RC Protocol Incompatibility:** Due to RC protocol incompatibility, the flight controller demanded serial SBUS input, whereas the receiver outputted parallel PWM. The PWM signal was captured by an ESP32 microcontroller and programmed to decode the pulses and output inverted 100,000 baud SBUS frames, allowing for reliable manual control.
- **Servo Voltage Fluctuations:** When controlling servos, we found that the servo was drawing a high amount of power, reducing the main voltage by 0.6 V. To remedy this, we switched the powering of the servo to an external 3 A BEC connected directly to the LiPo PDB, which eliminated the undesired voltage spikes.
- **MAVLink Deserialization Queuing:** Buffer overflows occurred when streaming telemetry data too fast, causing stuttering. Implementing thread-safe `asyncio.Queue` workers and asynchronous ACK verification in `mavlink_manager.py` ensured lossless telemetry streaming.

4.4.3 Limitations and Future Scope

There are a number of practical challenges and restrictions that provide opportunities for future innovation:

1. **GPS Positioning Variance:** The single-frequency NEO-M8N GPS exhibited a mean horizontal error of 4.3 m. Combining dual-frequency RTK GNSS would yield arrival errors of less than 0.1 m.
2. **RF Hardware Calibration:** Future versions should include SDR spectrum calibrated hardware in addition to passive Wi-Fi RSSI indicators.
3. **Active Mesh Validation:** Field trials should use active mesh nodes in the future, adhering to IEEE 802.11s or ESP-NOW standards, to observe post-deployment multi-hop communication capability and understand how the mesh network functions in real-life disaster scenarios.

4.5 Conclusion

In this project, an integrated design, development and experimental validation of a disaster response framework has been developed as the *UAV-Based Deployment System for Rapid Establishment of Emergency Communication Networks*.

The quadcopter we built showed great control and stability in flight, and was able to hold its altitude and navigate to points independently at good accuracy (approx. 4.3 m off) on its own. We also developed a communication system for the quadcopter to communicate with the base station, and it was actually quite successful, with all of the messages getting through, even from 250 meters away. Plus, the mechanism we had to carry a small payload, such as a communication capsule, performed well and landed pretty close to where it was supposed to — within 8 meters.

This custom Ground Control Station software, which uses an asynchronous Python FastAPI backend, Leaflet.js interactive mapping and SQLite persistence, creates an operational pipeline.

It supports ingestion of geo-referenced RSSI measurements, creation of spatial Inverse Distance Weighting (IDW) connectivity heatmaps in just 1.18 s, and categorization of communication tiers. The proposed multi-criteria site-scoring formulation for the automated approach was able to balance the connectivity deficit, backhaul reachability, and expected coverage recovery and provide optimized node coordinates.

Initial MATLAB/Simulink simulations showed that 16.3 percentage points more coverage was achieved after deploying the algorithms, compared with random placement.

These results, combined, provide a defensible, reproducible, technical basis linking aerial RF coverage reconnaissance with physical payload deployment, demonstrating the concept and practical potential of small autonomous UAVs for humanitarian disaster relief operations.

Appendix A

Appendix

A.1 Python GCS Backend Implementation

The custom Ground Control Station (GCS) backend is implemented in Python using the FastAPI framework. Listing A.1 provides the core survey collector state management, telemetry ingestion, and RESTful API endpoints from `backend/main.py`.

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import asyncio

app = FastAPI(title="UAV Emergency Communication GCS Backend", version="2.0")

class SurveySample(BaseModel):
    lat: float
    lon: float
    alt: float
    rssi: float

class RFCollector:
    def __init__(self):
        self.state = "IDLE"  # IDLE, MISSION_UPLOADED, RUNNING, COMPLETED
        self.samples: List[Dict[str, float]] = []
        self._lock = asyncio.Lock()

    async def ingest_telemetry(self, lat: float, lon: float, alt: float, rssi: float
        ):
        if self.state == "RUNNING":
            async with self._lock:
                sample = {"lat": lat, "lon": lon, "alt": alt, "rssi": rssi}
                self.samples.append(sample)
                return sample
        return None

    def start_scan(self):
        self.state = "RUNNING"

    def complete_scan(self):
        self.state = "COMPLETED"

    def reset(self):
```

```
        self.state = "IDLE"
        self.samples.clear()

collector = RFCollector()

@app.get("/api/rf-survey/state")
async def get_survey_state():
    return {
        "status": "success",
        "state": collector.state,
        "sample_count": len(collector.samples),
        "best_rssi": max([s["rssi"] for s in collector.samples], default=-100.0)
    }

@app.get("/api/rf-survey/data")
async def get_survey_data():
    return {"status": "success", "count": len(collector.samples), "data": collector.
        samples}

@app.post("/api/rf-survey/reset")
async def reset_survey():
    collector.reset()
    return {"status": "success", "message": "Survey buffer cleared and reset to IDLE
        "}

@app.post("/api/mission/start")
async def start_mission(mission_type: str = "survey"):
    if mission_type == "survey":
        collector.start_scan()
    return {"status": "success", "message": f"Mission {mission_type} started", "
        scan_state": collector.state}
```

LISTING A.1: Python FastAPI Backend – RF Survey Ingestion and API Endpoints

A.2 Python Multi-Criteria Site-Scoring Algorithm

The candidate site-scoring algorithm executed at the GCS backend evaluates potential communication node placement locations based on connectivity deficit, distance to anchor nodes, and projected coverage recovery (Listing A.2).

```
import math
from typing import List, Dict, Tuple, Any

def score_deployment_candidates(
    candidates: List[Tuple[float, float, float]],  # List of (x, y, rssi)
    gap_cells: List[Tuple[float, float]],           # Coordinates of degraded cells
        (RSSI < -75)
    anchors: List[Tuple[float, float]],             # Coordinates of surviving
        anchor stations
    w_C: float = 0.40, w_D: float = 0.30, w_A: float = 0.30,
    R_th: float = -75.0, R_cov: float = 40.0
) -> List[Dict[str, Any]]:
    """Evaluates and ranks candidate deployment locations based on C_k, D_k, and A_k
        ."""
    n_gaps = max(len(gap_cells), 1)
    scored_results = []

```

```
    for k, (cx, cy, cand_rssi) in enumerate(candidates, start=1):
        # 1. Connectivity Need Score (enforcing strict [0, 100] bounds)
        raw_deficit = (R_th - cand_rssi) / 30.0
        C_k = 100.0 * max(0.0, min(1.0, raw_deficit))

        # 2. Distance Suitability Score (Euclidean distance to closest anchor)
        min_anchor_dist = min([math.sqrt((cx - ax)**2 + (cy - ay)**2) for ax, ay in
            anchors])
        D_k = max(0.0, 100.0 - (min_anchor_dist / 5.0))

        # 3. Area Improvement Score (gap cells recovered within coverage radius
            R_cov)
        recovered = sum([1 for gx, gy in gap_cells if math.sqrt((cx - gx)**2 + (cy -
             gy)**2) <= R_cov])
        A_k = (recovered / n_gaps) * 100.0

        # Composite Score
        S_k = w_C * C_k + w_D * D_k + w_A * A_k

        scored_results.append({
            "candidate_id": k, "x": cx, "y": cy, "rssi": cand_rssi,
            "C_k": round(C_k, 1), "D_k": round(D_k, 1), "A_k": round(A_k, 1),
            "score": round(S_k, 1)
        })

    scored_results.sort(key=lambda item: item["score"], reverse=True)
    return scored_results
```

LISTING A.2: Python GCS Multi-Criteria Site-Scoring Function

A.3 Flight Controller Hardware and Actuator Interfaces

A.3.1 Servo Payload Release Channel Interface

The SG90 micro-servo drop latch is interfaced directly with the Edith IronCore F4 autopilot auxiliary output rail, as detailed in Table A.1.

TABLE A.1: Flight Controller Auxiliary Servo Channel Connection

Servo Lead	**Flight Controller Pin**	**Electrical Specification**
Signal (Orange/White)	AUX OUT 1 (Signal Rail)	50 Hz PWM (1000 μs locked / 2000 μs release)
Power (Red)	Dedicated BEC Output (+5 V)	Regulated 5.0 V DC from 3 A standalone BEC
Ground (Brown/Black)	Common Ground (GND)	Common system ground reference

Safety Note: Servo power is supplied from a dedicated 5 V Battery Eliminator Circuit (BEC) connected directly to the main LiPo power distribution board, rather than through the flight controller internal regulator, preventing inductive voltage dips during servo movement.

A.3.2 ESP32 AFHDS-to-SBUS Signal Converter Interface

To achieve compatibility between the 6-channel AFHDS radio receiver and the flight controller single inverted SBUS serial input port, an embedded ESP32 microcontroller performs real-time parallel-to-serial protocol conversion (Table A.2).

TABLE A.2: ESP32 AFHDS-to-SBUS Signal Converter Pin Connections

ESP32 Pin	Connected Subsystem	Functional Role
GPIO 13–18 (Inputs)	Receiver Channels 1–6 (PWM)	Parallel PWM pulse width capture via interrupts
GPIO 17 (TX2 Output)	Flight Controller SBUS RX	Serialized inverted SBUS frame (100 000 baud, 8E
VIN (+5 V)	Regulated 5 V Rail	System operating power
GND	Common System Ground	Common electrical reference

A.4 Preliminary MATLAB/Simulink Simulation Scripts

The following scripts were utilized during the initial simulation stage to validate 2.4 GHz log-distance propagation, evaluate IDW interpolation, and bridge GCS datasets into Simulink.

A.4.1 MATLAB Simulation Script: RF Propagation and IDW Heatmap

```
% --- Preliminary Simulation: RF Propagation & IDW Heatmap Generation ---
clear; clc; close all;

x_len = 100; y_len = 80; grid_res = 10;
[X_grid, Y_grid] = meshgrid(5:grid_res:x_len-5, 5:grid_res:y_len-5);
[n_rows, n_cols] = size(X_grid);
anchor_pos = [20, 25, 1]; % Surviving emergency base [x, y, z]

% 2.4 GHz Log-Distance Propagation Parameters
P_tx = 18.0; G_tx = 2.0; G_rx = 2.0; PL_d0 = 40.0; eta = 2.8; sigma = 3.0;

% Simulated UAV Raster Survey Trajectory
altitude = 20.0; track_spacing = 20.0; v_survey = 3.0; dt_sample = 0.5;
uav_samples = []; track_y = 10:track_spacing:70; direction = 1;

for ty = track_y
    tx_pts = (direction == 1) ? (10: (v_survey * dt_sample) : 90) : (90: -(v_survey
        * dt_sample) : 10);
    for tx = tx_pts
        d = max(sqrt((tx - anchor_pos(1))^2 + (ty - anchor_pos(2))^2 + (altitude -
            anchor_pos(3))^2), 1.0);
        PL = PL_d0 + 10 * eta * log10(d) + normrnd(0, sigma);
        rssi = P_tx + G_tx + G_rx - PL;
        uav_samples = [uav_samples; tx, ty, altitude, rssi];
    end
    direction = -direction;
end

% IDW Interpolation (p = 2.0)
RSSI_heatmap = zeros(n_rows, n_cols);
for r = 1:n_rows
```

```
    for c = 1:n_cols
        pt = [X_grid(r,c), Y_grid(r,c)];
        dists = sqrt((uav_samples(:,1) - pt(1)).^2 + (uav_samples(:,2) - pt(2)).^2);
        weights = 1.0 ./ (dists .^ 2.0);
        RSSI_heatmap(r,c) = sum(weights .* uav_samples(:,4)) / sum(weights);
    end
end
fprintf('Completed survey: %d samples. Heatmap: %d cells.\n', size(uav_samples, 1),
    n_rows*n_cols);
```

LISTING A.3: Preliminary MATLAB Script – RF Propagation and IDW Heatmap

A.4.2 MATLAB GCS Integration Bridge Script (`setupGCS.m`)

```
% --- MATLAB GCS Bridge: setupGCS.m ---
clear; clc;
gcs_url = 'http://127.0.0.1:8000/api/rf-survey/data';
options = weboptions('Timeout', 10, 'ContentType', 'json');

try
    fprintf('Connecting to GCS backend at %s...\n', gcs_url);
    response = webread(gcs_url, options);
    if isfield(response, 'data') && ~isempty(response.data)
        survey_records = response.data;
        lats = cellfun(@(s) s.lat, survey_records);
        lons = cellfun(@(s) s.lon, survey_records);
        alts = cellfun(@(s) s.alt, survey_records);
        rssi = cellfun(@(s) s.rssi, survey_records);
        lat0 = min(lats); lon0 = min(lons);
        x_m = (lons - lon0) * 111320 * cosd(lat0);
        y_m = (lats - lat0) * 111320;
        survey_dataset = [x_m, y_m, alts, rssi];
        assignin('base', 'survey_dataset', survey_dataset);
        fprintf('Loaded %d survey points into workspace as survey_dataset.\n',
            length(survey_records));
    end
catch ME
    warning('Failed to connect to GCS backend: %s', ME.message);
end
```

LISTING A.4: MATLAB GCS Bridge Script – `setupGCS.m`

A.4.3 Simulink Model Top-Level Subsystem Organization

The preliminary MATLAB/Simulink RF simulation model (`EmergencyNetwork.slx`) is organized into five interconnected functional subsystems:

1. **Subsystem 1 — UAV Trajectory Generator:** Discrete-time generator outputting survey coordinates $(x(t), y(t), z(t))$.
2. **Subsystem 2 — Terrestrial Node Environment:** Stores coordinates, transmit power, and antenna parameters of active ground nodes.
3. **Subsystem 3 — RF Propagation Subsystem:** Computes 3D distances, applies log-distance path loss, and introduces shadow fading.

4. **Subsystem 4 — Discrete Spatial Sampler:** Emulates periodic sampling, outputting geo-referenced measurement tuples at 2 Hz.
5. **Subsystem 5 — Heatmap and Decision Engine:** Executes IDW spatial estimation, link classification, gap detection, and candidate site scoring.

Bibliography

Biodata

Student 1: Ronie Samuel (23BLC1061)

Passport
Photo

Name: Ronie Samuel

Reg. No.: 23BLC1061

Programme: B.Tech (Electronics and Computer Engineering)

School: School of Electronics Engineering (SENSE)

Mobile No.: .....

E-mail: ronie.samuel2023@vitstudent.ac.in

Permanent Address: .....

.....

Student 2: Anmol Kapil (23BLC1229)

Passport
Photo

Name: Anmol Kapil

Reg. No.: 23BLC1229

Programme: B.Tech (Electronics and Computer Engineering)

School: School of Electronics Engineering (SENSE)

Mobile No.: .....

E-mail: anmol.kapil2023@vitstudent.ac.in

Permanent Address: .....

.....

Student 3: Garv Rawat (23BLC1058)

Passport
Photo

Name: Garv Rawat

Reg. No.: 23BLC1058

Programme: B.Tech (Electronics and Computer Engineering)

School: School of Electronics Engineering (SENSE)

Mobile No.: .....

E-mail: garv.rawat2023@vitstudent.ac.in

Permanent Address: .....

.....